

РЕГЛАМЕНТ ПРОВЕДЕНИЯ ФИНАЛЬНОГО ЭТАПА УЧЕБНОЙ ПРАКТИКИ

Модуль: Сопровождение и обслуживание программного обеспечения компьютерных систем (Часть 2)

Направление: Разработка отказоустойчивого микросервиса / утилиты с нуля

Продолжительность: 9 рабочих дней (34 пары / 68 академических часов)

1. ФОРМАТ КОМАНД И ПРАВИЛА РАЗРАБОТКИ

1. **Состав микро-команд:** Строго индивидуально или в микро-группах по **2 или максимум 3 человека**.
2. **Распределение ролей (для команд от 2 человек):** Внутри команды должно быть четкое разделение (например: Архитектор/ DevOps, Backend-разработчик, QA/Инженер по тестированию и логированию).
3. **Запрет на UI:** Веб-интерфейс (фронтенд) делать **запрещено**. Допускается: CLI (консольный интерфейс), REST API (с Swagger/ ReDoc), Telegram-бот или фоновый демон (сервис).
4. **Технологический стек:** Любой на выбор команды (Python, C#, PHP, JS/TS и т.д.), но с обязательным соблюдением стандартов промышленной разработки.

2. ГИТХАБ-ПОЛИТИКА (GIT FLOW)

Контроль работы будет вестись строго по истории активности в Git.

1. **Репозиторий:** Капитан команды (или индивидуальный разработчик) создает **абсолютно новый, пустой публичный репозиторий** на GitHub. Все участники добавляются как Collaborators.
2. **Ветки:** Ветка main (или master) - строго стабильная, код в неё попадает только через Pull Requests. Вся разработка ведется в ветках компонентов (например: feat/api, feat/logging, fix/bugs).
3. **Интенсивность коммитов:** Каждый участник обязан делать **минимум 1-2 осмысленных коммита каждый рабочий день**

практики. Если у студента за день нет коммитов, его работа за этот день аннулируется.

4. **Культура коммитов:** Сообщения должны быть информативными. Никаких fix, 111 или done. Применяем стандарт Conventional Commits (например: feat(auth): added jwt validation middleware или docs(readme): updated env variable map).

3. ОБЯЗАТЕЛЬНЫЕ ТЕХНИЧЕСКИЕ ТРЕБОВАНИЯ (С отсылкой к 1-й части практики)

Продукт пишется с нуля, но обязан **сразу** включать в себя лучшие практики и инструменты, которые студенты освоили на прошлых спринтах:

- **Чистая архитектура и Линтеры (Из Спринта 1):** Код должен быть разбит на слои (маршруты, бизнес-логика, доступ к данным). В репозиторий должен быть интегрирован инструмент статического анализа (линтер под выбранный стек). Код на защите должен проходить проверку линтером без критических ошибок.
- **Изоляция секретов (Из Спринта 1):** В коде не должно быть ни одного захардкоженного токена, пароля или ключа. Все настройки, API-ключи и доступы выносятся строго в переменные окружения (.env). В репозитории оставляется только безопасный файл .env.example.
- **Документирование систем (Из Спринта 2):** В корне проекта обязателен подробный README.md. Он должен содержать: описание утилиты, архитектурную схему, карту переменных окружения с примерами и гайд по запуску. Для API обязательна интерактивная документация (Swagger/OpenAPI).
- **Сквозное логирование (Из Спринта 3):** Приложение должно подробно логировать свою работу. Логи разделяются на потоки (INFO для штатных событий, ERROR/CRITICAL для сбоев). Логирование должно покрывать критические узлы: входящие запросы, интеграции с внешними API и обработку ошибок. Логи должны выводиться в консоль (для Docker) и писаться в ротлируемые файлы.
- **Контейнеризация (Базовое требование модуля):** Наличие корректного Dockerfile и docker-compose.yml (если используется

БД или Redis) в корне проекта. Сервис должен разворачиваться и запускаться одной командой: `docker compose up --build`.

4. КАЛЕНДАРЬ ОТЧЕТНОСТИ (9 ДНЕЙ ПРАКТИКИ)

Отчетность делится на **3 контрольные точки (каждые 3 дня)** для проверки прогресса в Git, а в финале сдается **итоговый отчет** и проходит **живой экзамен**.

Точка 1: Проектирование и Старт (Конец 3-го дня 19.06)

- **Что проверяется:** Наличие репозитория, файла README.md с описанием и распределением ролей (если это команда). Наличие архитектурного плана или UML-схем (Use Case / Sequence). Настроенное базовое окружение и первая рабочая кодовая база.
- **Формат:** Проверка репозитория по ссылкам (оцениваются коммиты за 1, 2, 3 дни).

Точка 2: Ядро и Инфраструктура (Конец 6-го дня 24.06)

- **Что проверяется:** Реализована основная бизнес-логика утилиты. Настроена изоляция секретов через .env. Внедрен модуль логирования (Спринт 3). Написан Dockerfile для контейнеризации приложения.
- **Формат:** Проверка коммитов (за 4, 5, 6 дни)

Точка 3: Финализация и Стабилизация (Конец 9-го дня 29.06)

- **Что проверяется:** Код полностью написан и протестирован, исправлены синтаксические предупреждения по линтеру (Спринт 1). Документация в README.md и Swagger полностью актуализирована (Спринт 2). Система работает стабильно в Docker-контейнере.
- **Формат:** Загрузка черновиков отчетов. Запрет на внесение крупных изменений в код (период Code Freeze).

10 День: Итоговая отчетность и Экзамен

Каждая команда (или индивидуальный разработчик) сдает **один отчет** (в формате PDF/Word) и защищает проект очно.

5. РЕГЛАМЕНТ ПРОВЕДЕНИЯ ЭКЗАМЕНА (10 ДЕНЬ)

Экзамен проходит в очном формате и состоит из двух этапов:

- **Этап 1. Живой показ и защита сервиса (Презентация):**
Студенты разворачивают свой сервис "из нуля" в Docker-контейнере одной командой. Демонстрируют выполнение целевой функции утилиты. Показывают логирование ошибок в реальном времени при искусственном сбое.
- **Этап 2. Блиц-опрос по коду:** Индивидуальные вопросы каждому участнику по архитектуре кода, механизму изоляции секретов и настройкам контейнеризации